

NAME : SHAH FAISAL

ROLL NO : 23I-0058

SECTION : BSE – 4A

Operating Systems Lab 09

BSSE SPRING 2026

LAB 09

Lab09 task1 · C

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
```

```
int main() {
    // Open (or create) a file for writing
    int fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) {
        perror("open");
        return 1;
    }
```

```
    // Step 1: Use dup() to duplicate fd, store in fd_dup
    int fd_dup = dup(fd);
    if (fd_dup < 0) {
        perror("dup");
        return 1;
    }
```

```
    // Step 2: Save original stdout so we can restore it later
    int saved_stdout = dup(1);
```

```
    // Step 3: Use dup2() with fd_dup to redirect stdout (fd 1) to the file
    dup2(fd_dup, 1);
```

```
    // This printf goes to output.txt, NOT the terminal
    printf("Hello World\n");
    printf("This line is written to the file via dup + dup2!\n");
    fflush(stdout); // Important: flush buffered output before restoring
```

```
    // Step 4: Restore original stdout
    dup2(saved_stdout, 1);
    close(saved_stdout);
```

```
    // Now this prints to the terminal again
    printf("Back to terminal! Check output.txt for the redirected output.\n");
```

```
    close(fd);
    close(fd_dup);
```

```
return 0;  
}
```

```
shah@DESKTOP-02Q2MPC:~$ gcc Lab09_task1.c -o task1 && ./task1  
Back to terminal! Check output.txt for the redirected output.
```

```
#include <stdio.h>  
#include <fcntl.h>  
#include <unistd.h>
```

```
int main() {  
    // Create input.txt with some content to read  
    FILE *setup = fopen("input.txt", "w");  
    fprintf(setup, "Hello, World! This came from input.txt\n");  
    fclose(setup);  
  
    // Open input.txt for reading  
    int fd = open("input.txt", O_RDONLY);  
    if (fd < 0) {  
        perror("open");  
        return 1;  
    }  
  
    // Redirect stdin (fd 0) to input.txt using dup2  
    dup2(fd, 0);  
    close(fd); // Original fd no longer needed  
  
    char buffer[256];  
  
    // fgets reads from stdin, which now points to input.txt  
    fgets(buffer, sizeof(buffer), stdin);  
  
    printf("Read from file via stdin: %s", buffer);  
  
    return 0;  
}
```

```
shah@DESKTOP-02Q2MPC:~$ gcc Lab09_task2.c -o task2 && ./task2  
Read from file via stdin: Hello, World! This came from input.txt
```

```
#include <stdio.h>  
#include <fcntl.h>
```

```
#include <unistd.h>

int main() {
    // Open a file to redirect stderr into
    int fd = open("error_log.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) {
        perror("open");
        return 1;
    }

    // Redirect stderr (fd 2) to error_log.txt
    dup2(fd, 2);
    close(fd);

    // This goes to the terminal (stdout is unchanged)
    printf("This is normal output on the terminal.\n");

    // These error messages go to error_log.txt instead of terminal
    fprintf(stderr, "ERROR: Something went wrong!\n");
    fprintf(stderr, "ERROR: File not found (simulated error).\n");
    perror("perror also goes to stderr");

    printf("Check error_log.txt to see the error messages.\n");

    return 0;
}
```

```
shah@DESKTOP-02Q2MPC:~$ gcc Lab09_task3.c -o task3 && ./task3
This is normal output on the terminal.
Check error_log.txt to see the error messages.
```

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int pipefd[2]; // pipefd[0] = read end, pipefd[1] = write end

    if (pipe(pipefd) < 0) {
        perror("pipe");
        return 1;
    }
}
```

```
pid_t pid = fork();

if (pid < 0) {
    perror("fork");
    return 1;
}

if (pid == 0) {
    // ===== CHILD PROCESS =====

    // Close the read end (child only writes)
    close(pipefd[0]);

    // Save original stdout
    int saved_stdout = dup(1);

    // Redirect stdout to the write end of the pipe
    dup2(pipefd[1], 1);
    close(pipefd[1]);

    // printf now writes into the pipe (not the terminal)
    printf("Message from child process!\n");
    printf("Hello from the child via pipe!\n");
    fflush(stdout); // Flush before restoring

    // Restore stdout (optional but good practice)
    dup2(saved_stdout, 1);
    close(saved_stdout);
} else {
    // ===== PARENT PROCESS =====

    // Close the write end (parent only reads)
    close(pipefd[1]);

    // Wait for child to finish writing
    wait(NULL);

    char buffer[256];
    ssize_t bytes;
```

```
printf("Parent received from child:\n");

// Read all data from the pipe and print it
while ((bytes = read(pipefd[0], buffer, sizeof(buffer) - 1)) > 0) {
    buffer[bytes] = '\0';
    printf("%s", buffer);
}

close(pipefd[0]);
}

return 0;
}
```

```
This message is shown once a day. To disable it please create the
/home/shah/.hushlogin file.
shah@DESKTOP-02Q2MPC:~$ gcc Lab09_task1.c -o task1 && ./task1
Back to terminal! Check output.txt for the redirected output.
shah@DESKTOP-02Q2MPC:~$ gcc Lab09_task2.c -o task2 && ./task2
Read from file via stdin: Hello, World! This came from input.txt
shah@DESKTOP-02Q2MPC:~$ gcc Lab09_task2.c -o task2 && ./task2
ab09_task3.c -o task3 && ./task3Read from file via stdin: Hello, World! This came from input.txt
shah@DESKTOP-02Q2MPC:~$ gcc Lab09_task3.c -o task3 && ./task3
This is normal output on the terminal.
Check error_log.txt to see the error messages.
shah@DESKTOP-02Q2MPC:~$ gcc Lab09_task4.c -o task4 && ./task4
Parent received from child:
Message from child process!
Hello from the child via pipe!
shah@DESKTOP-02Q2MPC:~$
```